AIM- To configure an external interrupt (EXTI) for a user button and use a binary semaphore to synchronize an LED control task.

APPARATUS- STM32 Nucleo-F446RE development board, USB Type-A to Mini-B cable, STM32 CUBE IDE, STM32 CubeMX.

THEORY

This experiment explores the fundamental RTOS concept of **Deferred Interrupt Processing**, where time-consuming application logic is moved out of an Interrupt Service Routine (ISR) and into a dedicated task to maintain system responsiveness. In a real-time environment, keeping ISRs extremely short is critical to ensure the processor can handle multiple hardware events without missing deadlines.

The LED control task in this experiment involves blinking the LED **five times** with ON/OFF delay of 250 ms. The application flow for this experiment is given below:

1. Hardware-to-Software Signaling

The process begins with the **External Interrupt (EXTI)** controller. When the user button is pressed, the hardware detects a signal edge and triggers an interrupt. The CPU immediately pauses its current task to execute the associated ISR. However, instead of toggling the LED directly within the ISR—which would be a "bare-metal" approach—the ISR performs a single, rapid action: it **releases (gives)** a Binary Semaphore.

2. Binary Semaphore Synchronization

A **Binary Semaphore** acts as a synchronization primitive with two states: Available (1) and Unavailable (0). In this experiment, the semaphore is initialized to 0.

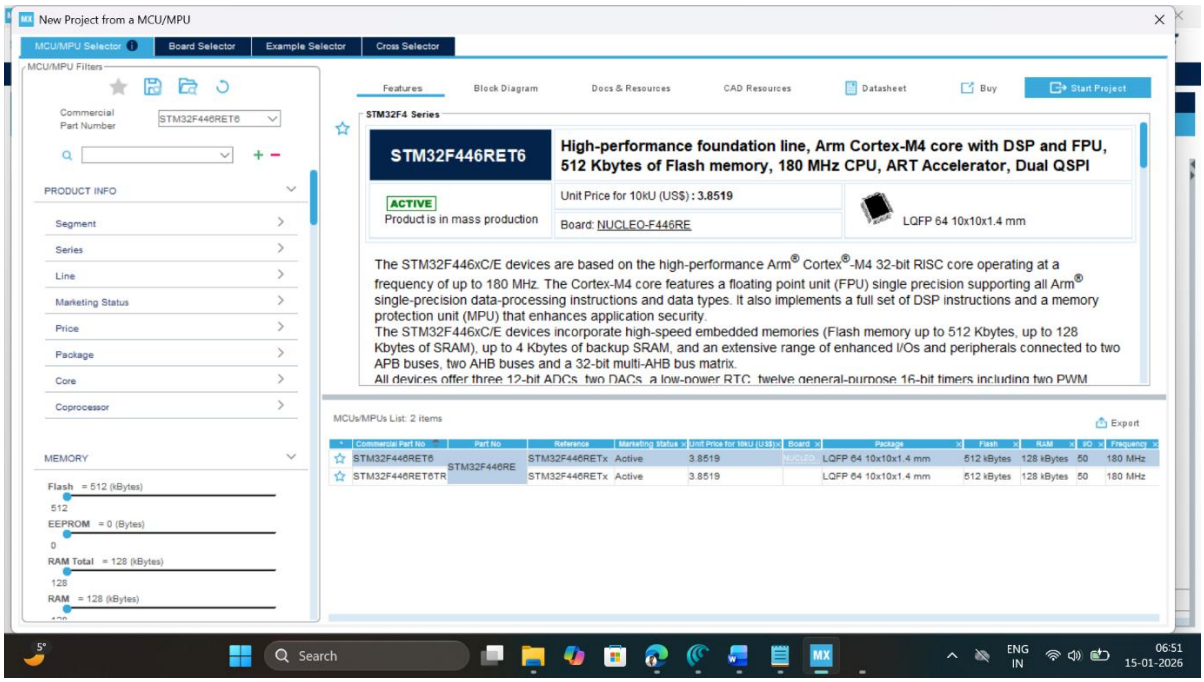
- **The Consumer:** The LED Control Task calls `osSemaphoreAcquire`. Since the count is 0, the task enters the **Blocked** state.
- **The Producer:** When the button ISR calls `osSemaphoreRelease`, the count increments to 1.

Task Preemption and Execution

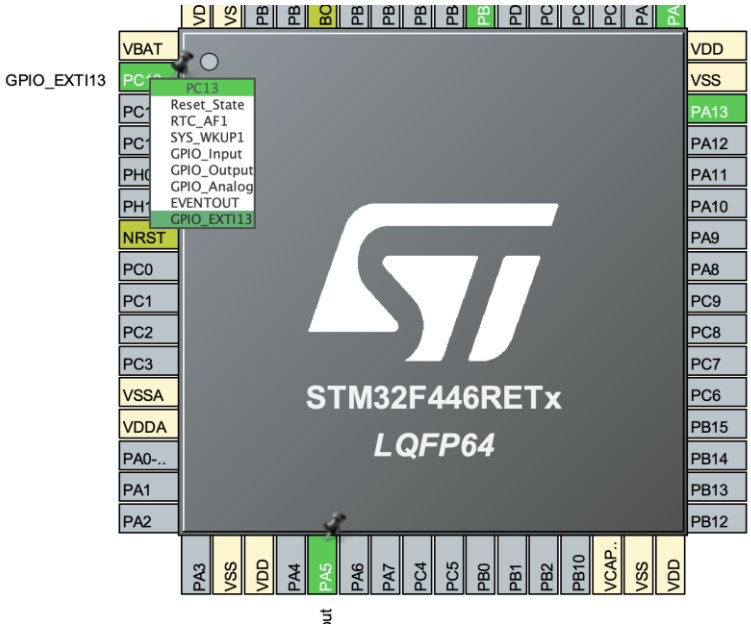
The moment the semaphore is released, the RTOS scheduler identifies that the LED task is now in the **Ready** state. If the LED task has a higher priority than the currently running task, the scheduler performs a **context switch**. The LED task "takes" the semaphore (resetting the count to 0), toggles the LED five times, and then loops back to block again, awaiting the next hardware trigger. This ensures the task responds nearly instantly to the button press while allowing the CPU to remain idle or handle other tasks in the interim.

PROCEDURE

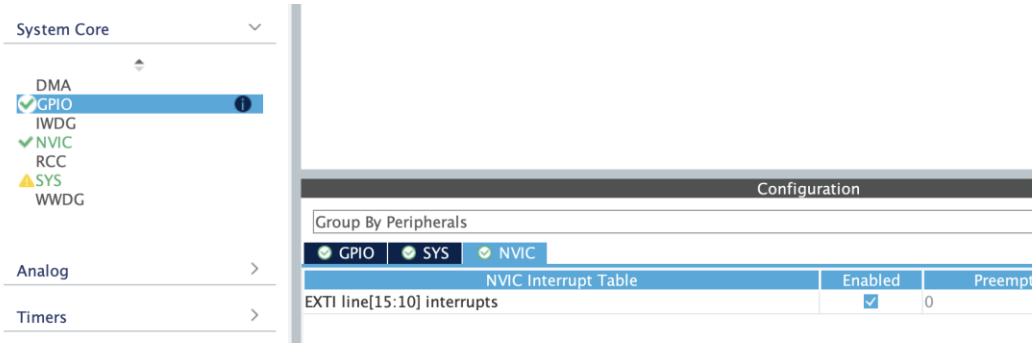
1. Open CubeMX and Click on ACCESS TO MCU SELECTOR.



2. Write and select the microcontroller as you want to use and then click start the project. Select commercial part number STM32L446RE.
3. In system Core select GPIO and right click on pin PA5 and to make it as GPIO output pin.
4. Right click on pin PC13 to enable the interrupt on the pin.

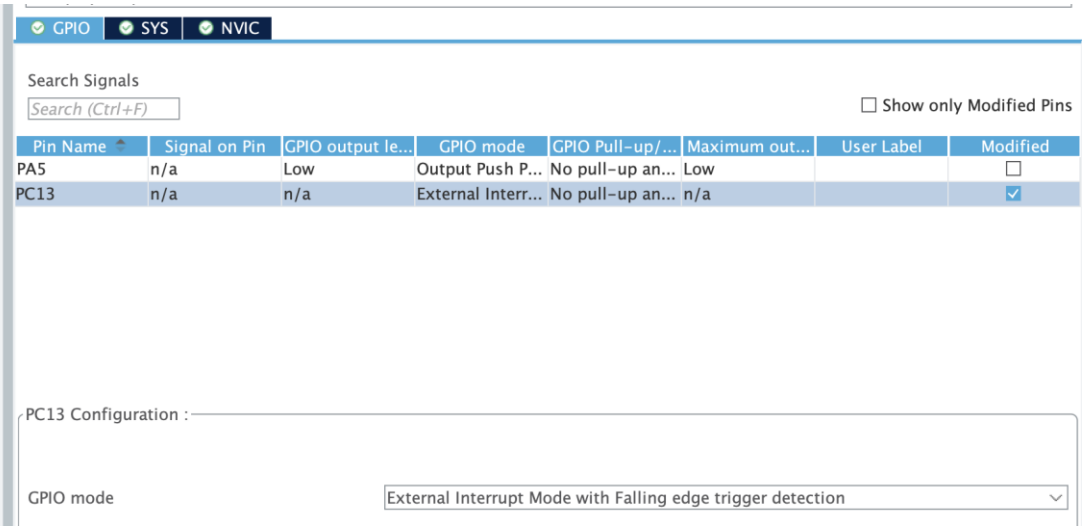


5. **Interrupt Configuration:** Go to System Core → GPIO and select NVIC tab.



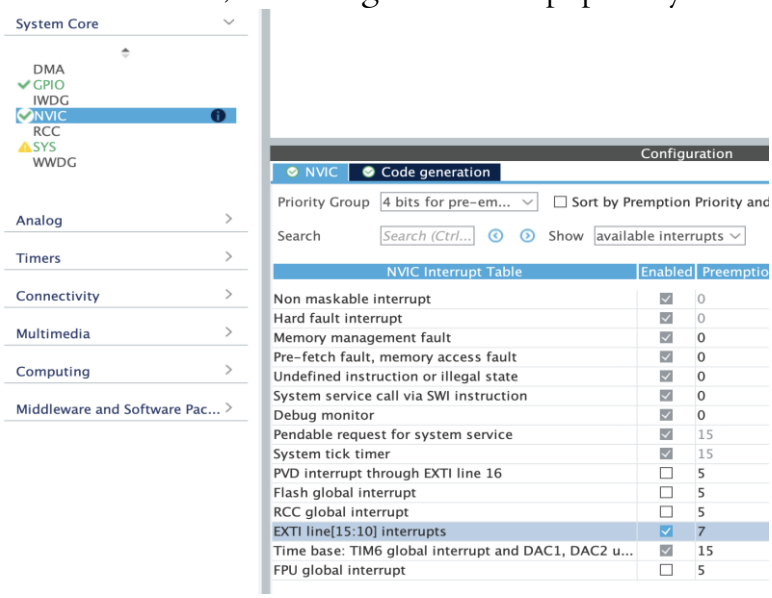
Enable the EXTI line[15:10] interrupt.

6. On the GPIO tab**, select 'Falling Edge Trigger Detection'** mode for interrupt

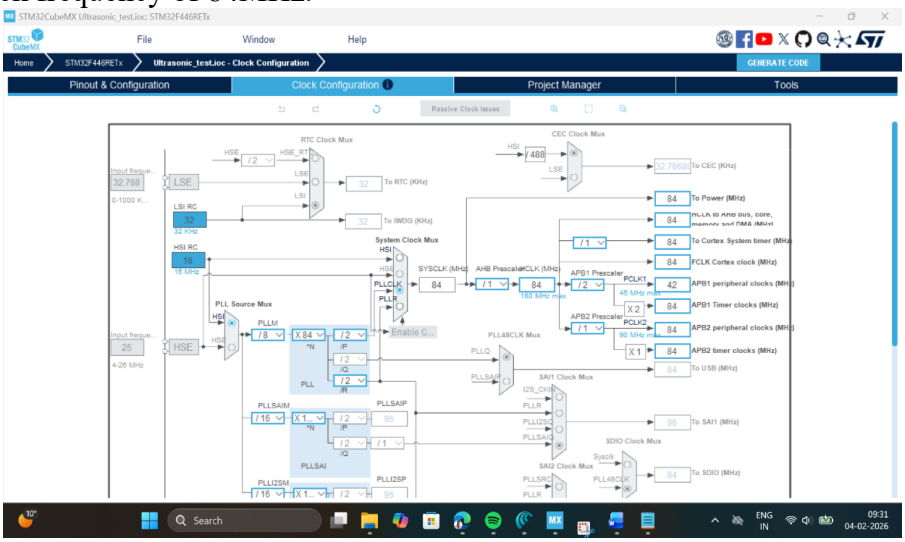


generation on GPIO pin PC13.

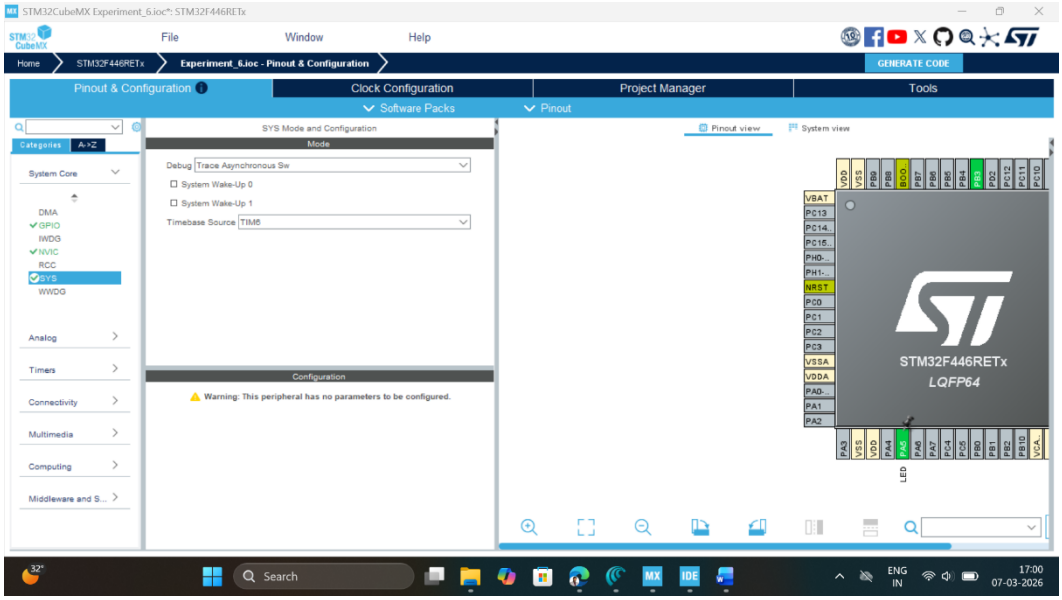
7. Select System Core → NVIC, and change the interrupt priority from 0 to 7.



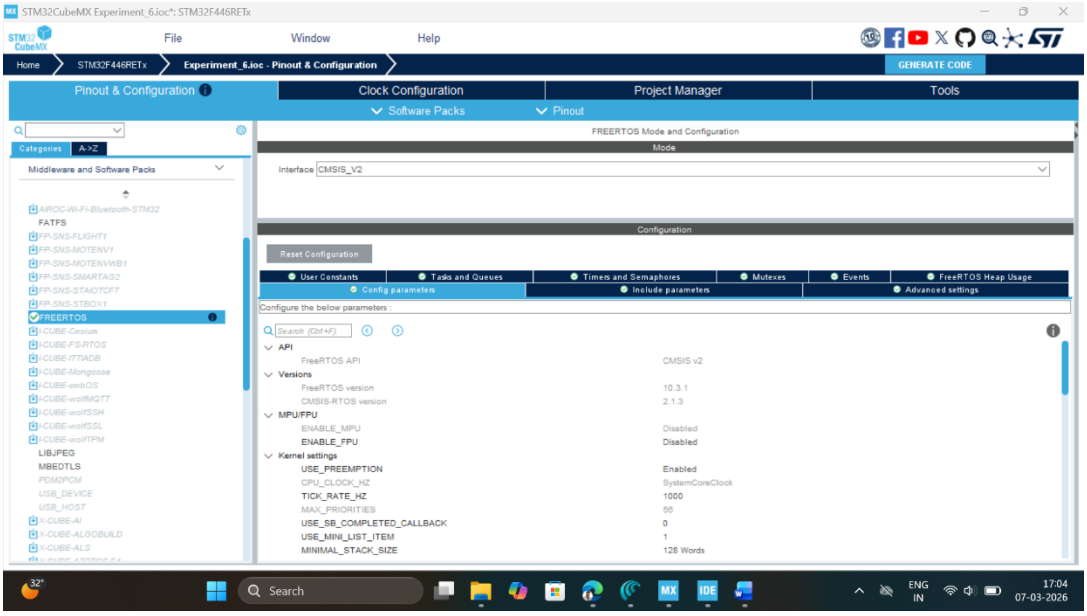
8. **Clock Configuration:** Configure clock as shown below to get maximum internal clock frequency of 84MHz.



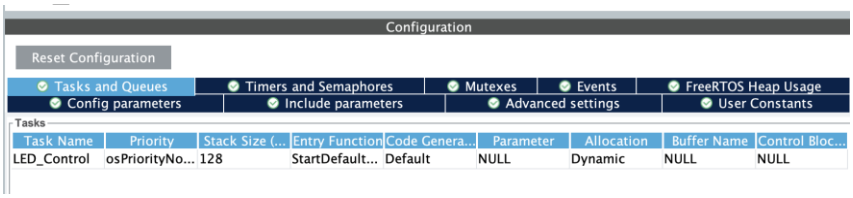
9. **SYS Mode:** Click on SYS mode and select debug as *Trace Asynchronous Sw*, and Timebase source to **TIM6.**



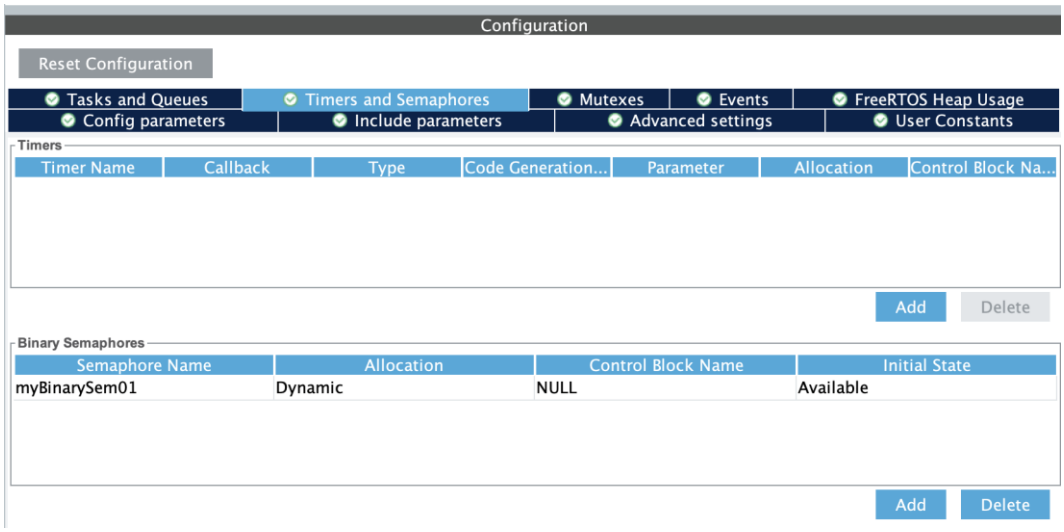
10. **FreeRTOS Configuration:** Click on Middleware and Software Pack, then select FreeRTOS software pack, which is already available with STMCube Mx. Select interface as **CMSIS_V2.**



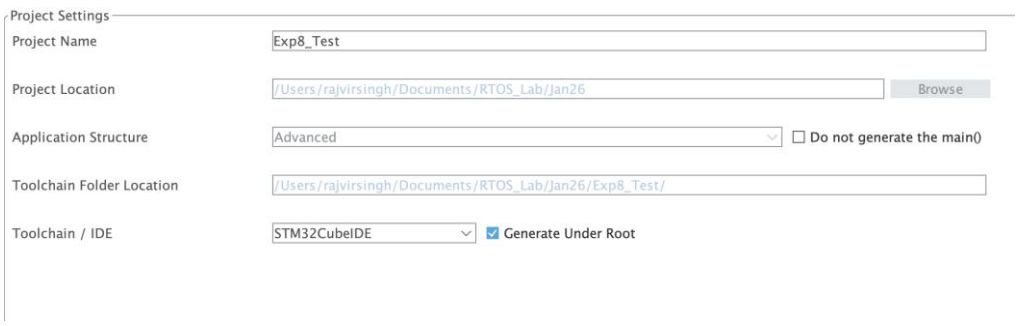
11. Click on Task & Queue Tab of configuration settings and rename the default task as LED_Control.



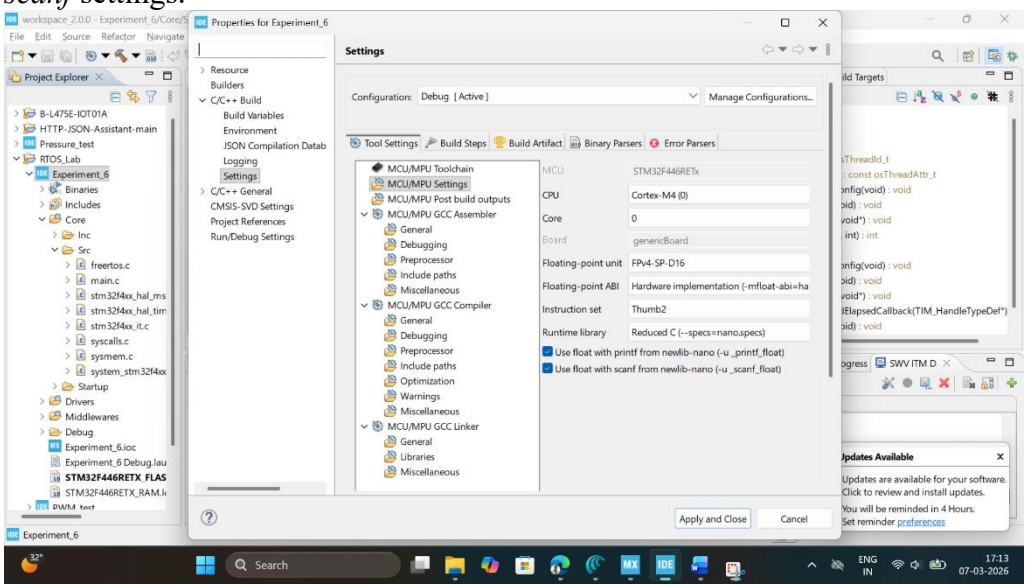
12. Click on TImers and Semaphores tab and ‘Add’ a binary semaphore as shown below:



13. Write Project name and select IDE.

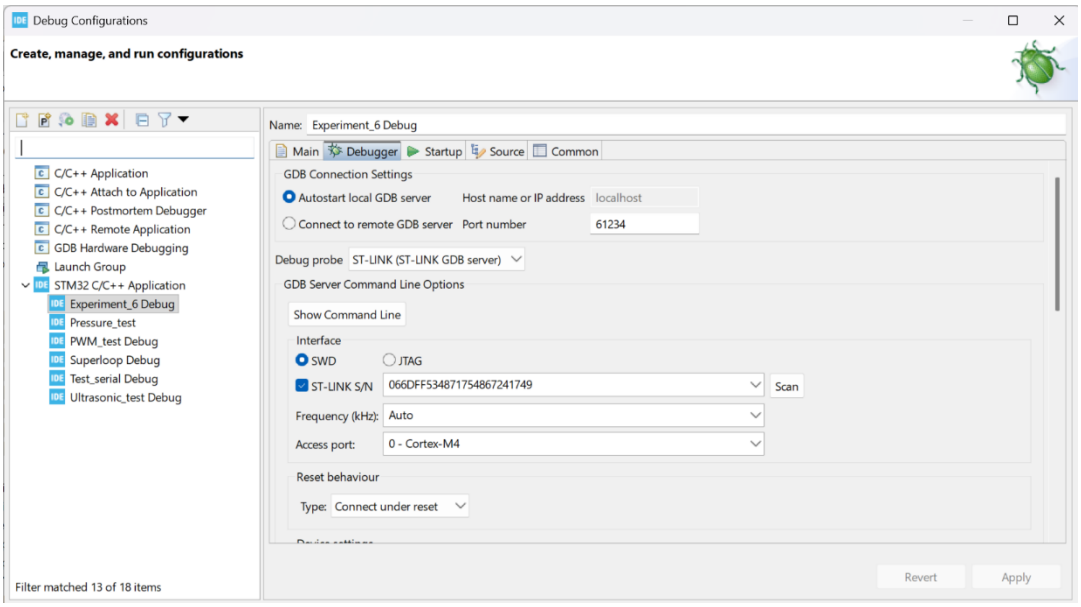


14. Click on generate code.
15. Click on the open project.
16. Click 'OK' on the generated popup showing "successfully imported the project on the workspace" and the project will be successfully added to the cube IDE.
17. Open the main source code on Cube IDE by clicking on **main.c**
18. Click on project properties, go to C/C++ Build settings check box for *printf* and *scanf* settings.



19. Now click on Debug icon and configure debugger for SWV ITM Trace
20.

21. Click on Debugger configuration, select debugger console, and click on ST-LINK S/N and then scan, as we attach a STM32F446RE development board, it will automatically pick up ST-LINK id.



22. Next, select the Serial Wire Viewer (SWV) and check box on enable button and make sure to pass on the right core clock frequency which is set-up at per point 4 here i.e. 84MHz. Then click on Apply button and close this window.
23. After configuration write following instructions in the **main.c** file and compile the program and ensure there will be zero error after compiling.

```
/* USER CODE BEGIN Includes */

#include <stdio.h>

/* USER CODE END Includes */
```

Write the function to enable SWV ITM Console to trace the output of LED_Control task

```
/* USER CODE BEGIN 0 */
// Retarget fwrite to ITM
int _write(int file, char *ptr, int len) {
   for (int i = 0; i < len; i++) {
      ITM_SendChar(*ptr++);
   }
   return len;
}
/* USER CODE END 0 */
```

Edit the LED_Control task function as given below to toggle the inbuilt LED five times with ON/OFF delay of 250ms.

```
void StartDefaultTask(void *argument)
{
        uint8_t i;
        /* Infinite loop */
        for(;;)
        {
                if (osSemaphoreAcquire(myBinarySem01Handle, 100) == osOK){
                        printf("Inside LEDControl Task\n");
                        i = 0;
                        while(i < 10){
                                HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_5);
```

```
                                HAL_Delay(250);
                                i = i + 1;
                        }
                }
        }
}
```

Also, add the following Interrupt Service Routine to your main.c file as shown below:

```
/**
  * @note   This function is called  when EXTI interrupt took place on line 13, inside
  * HAL_GPIO_EXTI_IRQHandler().
  */
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin){
        osSemaphoreRelease(myBinarySem01Handle);//Give Semaphore
}
```

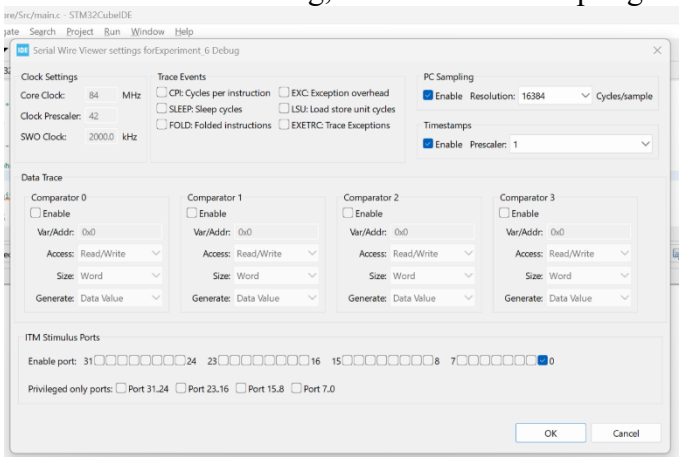
Change to the initial count of binary semaphore from 1 to 0 by modifying the second parameter of the following statement in your code:

myBinarySem01Handle = osSemaphoreNew(1, 0, &myBinarySem01_attributes);

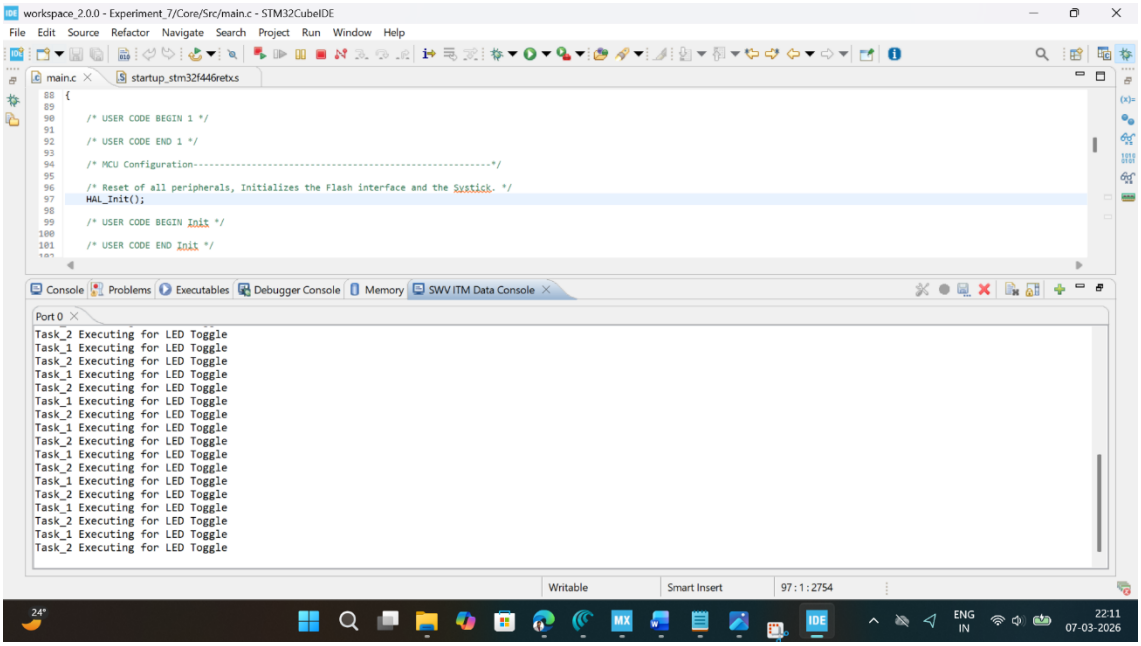
Please Note: This change is also required every time code is re-generated in CubeMX

.

24. Now, click on Window tab, select show view other select SWV ITM Data Console.
25. Now click on the Debug button, start debugging this main.c file and click on switch to change the perspective to debugger.
26. Click on SWV ITM Data Console setting, enable the PC sampling and select port 0.



27. Click on Start Trace (Red Dot) on SWV ITM Data Console tab and then click on resume button to start executing the main program. Now you can see the message on Port 0 and LED Toggle on STM32F446RE development board.



OBSERVATION:

S. No	Query	Response
1	What is the state of LED_Control task when application starts running?	At the start of the application, the LED_Control task remains in the **Blocked state** because it is waiting to acquire the binary semaphore.
2	What is the state of LED_Control task when button is pressed?	When the button is pressed, the external interrupt releases the binary semaphore, causing the LED_Control task to move from the **Blocked state to the Ready/Running state**.
3	Is LED_Control task running continuously after the button is pressed?	No, the LED_Control task does not run continuously. It executes only after receiving the semaphore, performs the LED blinking operation, and then waits again for the next semaphore event.
4	What is the priority level of the EXTI interrupt line?	The EXTI interrupt line generally has a **higher priority than normal tasks** so that button press events can be serviced immediately. The exact priority depends on the NVIC configuration in STM32CubeMX.

RESULT

The binary semaphore was successfully used to synchronize the LED_Control task with the external button press event. Whenever the button was pressed, an interrupt was generated that released the binary semaphore. The LED_Control task acquired the semaphore and executed the LED blinking operation. This experiment demonstrated effective task synchronization and event handling in FreeRTOS using binary semaphores.

Reflection Summary

1. Why LED_Control task is not running continuously after the button is pressed?

The LED_Control task is not running continuously because it is designed to execute only when the binary semaphore becomes available. After completing the LED blinking operation, the task again waits for the semaphore using osSemaphoreAcquire(). Since the semaphore is released only during a button press interrupt, the task remains blocked most of the time, saving CPU resources and improving system efficiency.

2. What is a breakpoint and how it can be useful in the debug mode?
A breakpoint is a debugging feature that temporarily pauses program execution at a specified line of code. It helps developers:

- Observe variable values.
- Analyze program flow step-by-step.
- Detect logical and runtime errors.
- Monitor task switching and RTOS behavior.

In debug mode, breakpoints are very useful for understanding how tasks, interrupts, and semaphores operate in real time.

3. Explain the working of the following piece of code in the program?

```
for(;;)
{
  if(osSemaphoreAcquire(myBinarySem01Handle, 100) == osOK){
    i = 0;
    while(i<10){
      HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_5);
      osDelay(250);
      i = i + 1;
    }
    printf("Inside LEDControl Task\n");
  }
}
```

- for(;;) creates an infinite loop so the task runs continuously.
- osSemaphoreAcquire(myBinarySem01Handle, 100) waits for the binary semaphore for 100 ticks.
- If the semaphore is successfully acquired (osOK):
  - Variable i is initialized to 0.
  - LED connected to GPIOA PIN5 toggles 10 times.
  - osDelay(250) creates a 250 ms delay between toggles.
  - After blinking completes, the message "Inside LEDControl Task" is printed.
- After execution, the task again waits for the semaphore.

Thus, the LED blinks only when the button press interrupt releases the semaphore.

4. Test the application by replacing the above code with the following code snippet and reflect on the change in the behavior of the application?

```
for(;;)
 {
    osSemaphoreAcquire(myBinarySem01Handle, 100);
    i = 0;
    while(i<10){
      HAL_GPIO_TogglePin(GPIOA,GPIO_PIN_5);
      osDelay(250);
      i = i + 1;
    }
    printf("Inside LEDControl Task\n");

 }
```

Change in Behavior

In this modified code, the return value of osSemaphoreAcquire() is not checked. Therefore:

- Even if the semaphore is not successfully acquired within 100 ticks, the LED blinking code still executes.
- As a result, the LED may blink continuously or periodically even without pressing the button.
- The synchronization between the button press event and the task execution is lost.

Conclusion

The original code is better because it checks whether the semaphore was successfully acquired before executing the LED operation. This ensures proper event-driven synchronization and prevents unintended task execution.